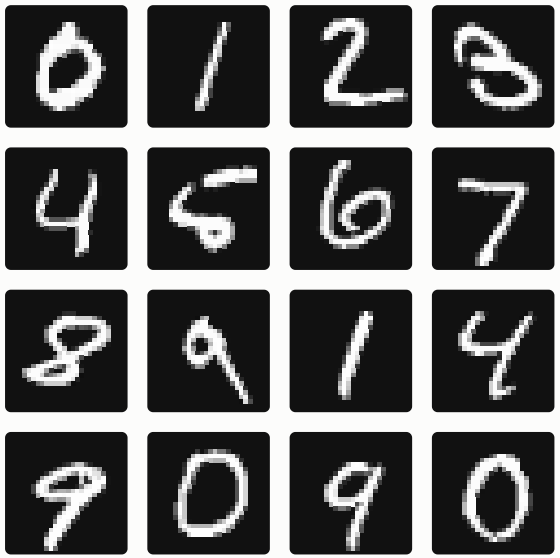
MNIST Accelerator

An FPGA neural-network accelerator that recognises handwritten digits

Group of 4 · Nexys A7 board (Artix-7) · 12 October to 13 December 2026 · Verilog + Python

16 real test images stored on the FPGA



Artix-7 FPGA

784 → 32 → 10 network
8-bit integer maths
one MAC unit, 100 MHz

What the hardware should answer

0	1	2	3
4	6	6	7
8	9	1	4
9	0	9	0

15 of 16 correct. The 5 is read as a 6, and the hardware must make exactly the same mistake.

96.6%
accuracy on 10,000 test images, after 8-bit conversion

25,408
multiply-adds to recognise one image

≈254 µs
per image with one MAC unit at 100 MHz

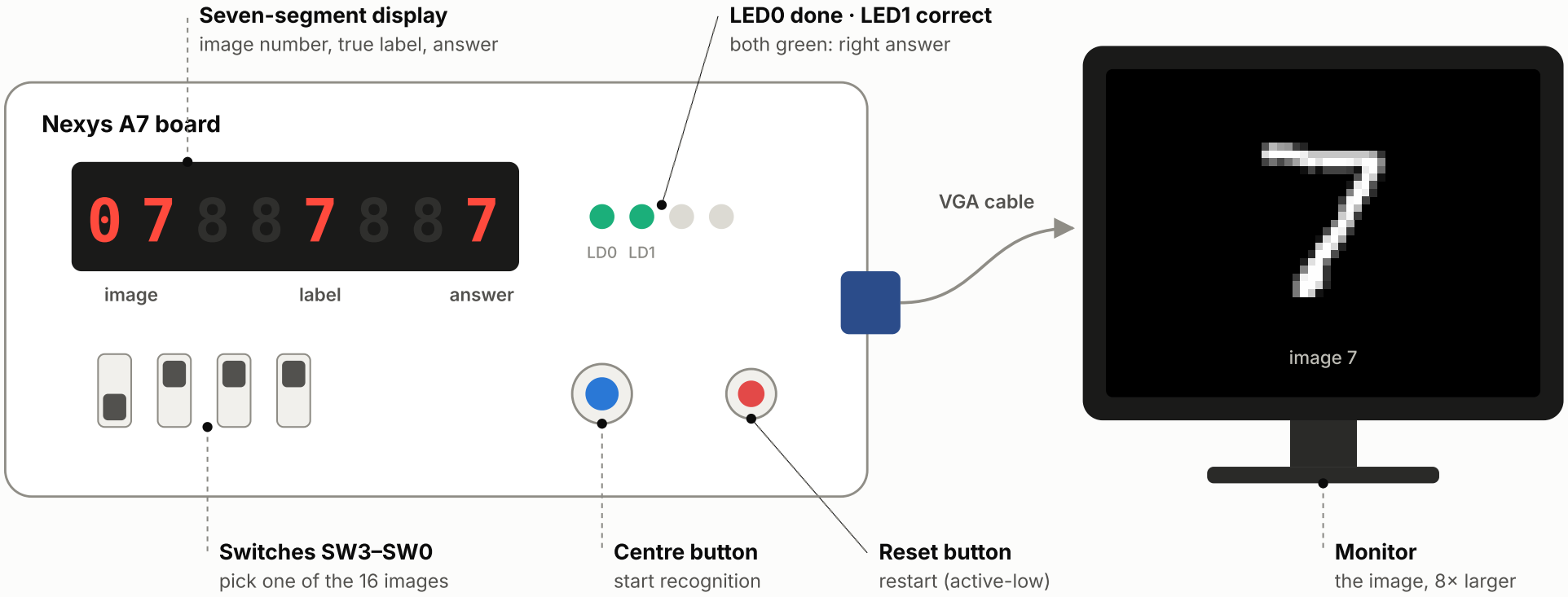
62 of 72 h
planned team hours (4 people × 2 h × 9 weeks)

29 Nov
working version tagged in Git

WHAT YOU WILL DEMO

Pick an image with the switches, press start, read the answer

The board recognises the digit by itself. Python is only used beforehand, to train the network and prepare the files.



How it works

1 Train in Python
60,000 MNIST images, network 784 → 32 → 10

→

2 Convert to 8-bit
weights −127…127, pixels 0–127

→

3 Export .hex files
weights, biases, 16 images, expected answers

→

4 FPGA computes
25,408 multiply-adds in ≈254 µs

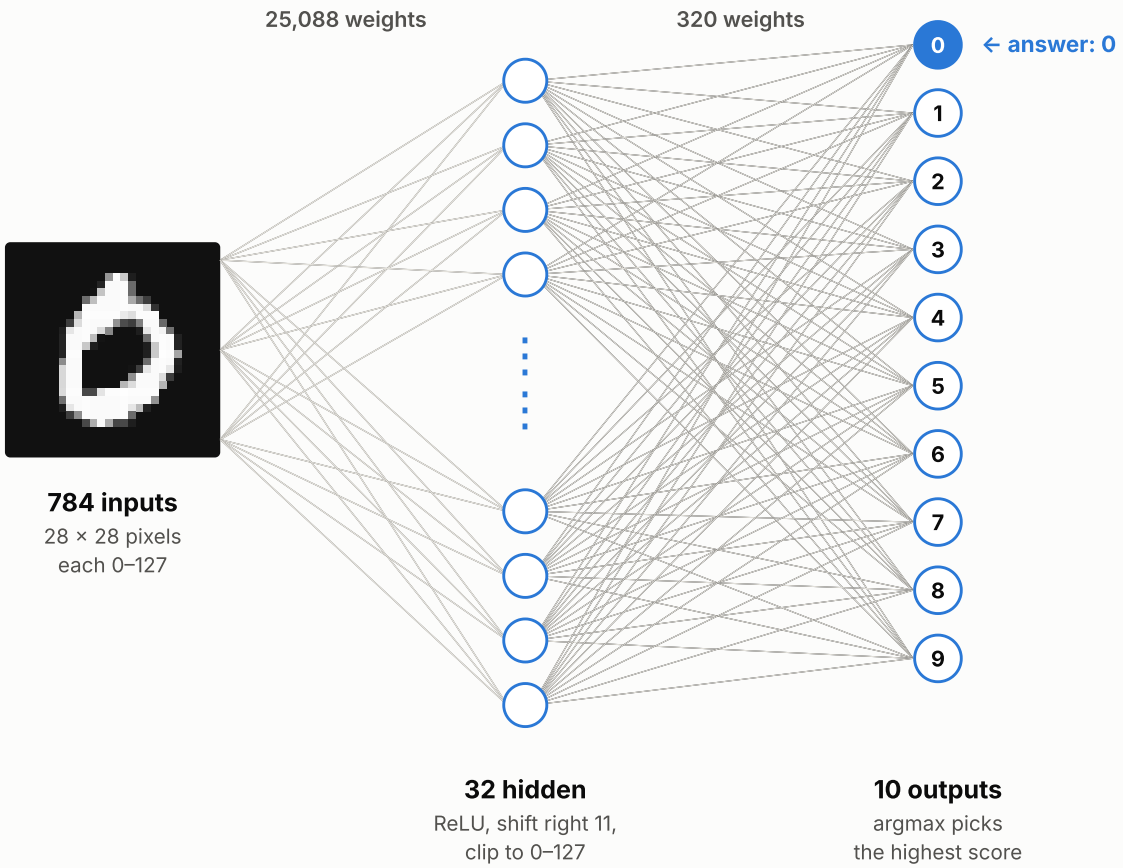
→

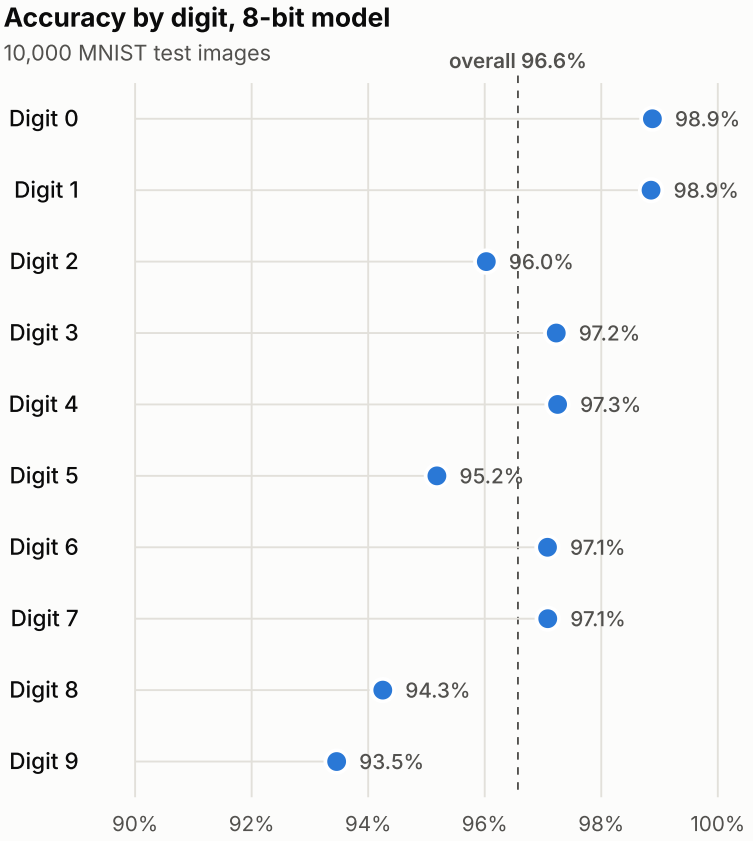
5 Show the answer
VGA image + digit on the seven-segment display

THE NEURAL NETWORK

A small network, all in 8-bit integers

Trained in Python on 60,000 images. Converting it to 8-bit integers kept the accuracy at 96.6%, the same as floating point, so the simple hardware loses nothing.

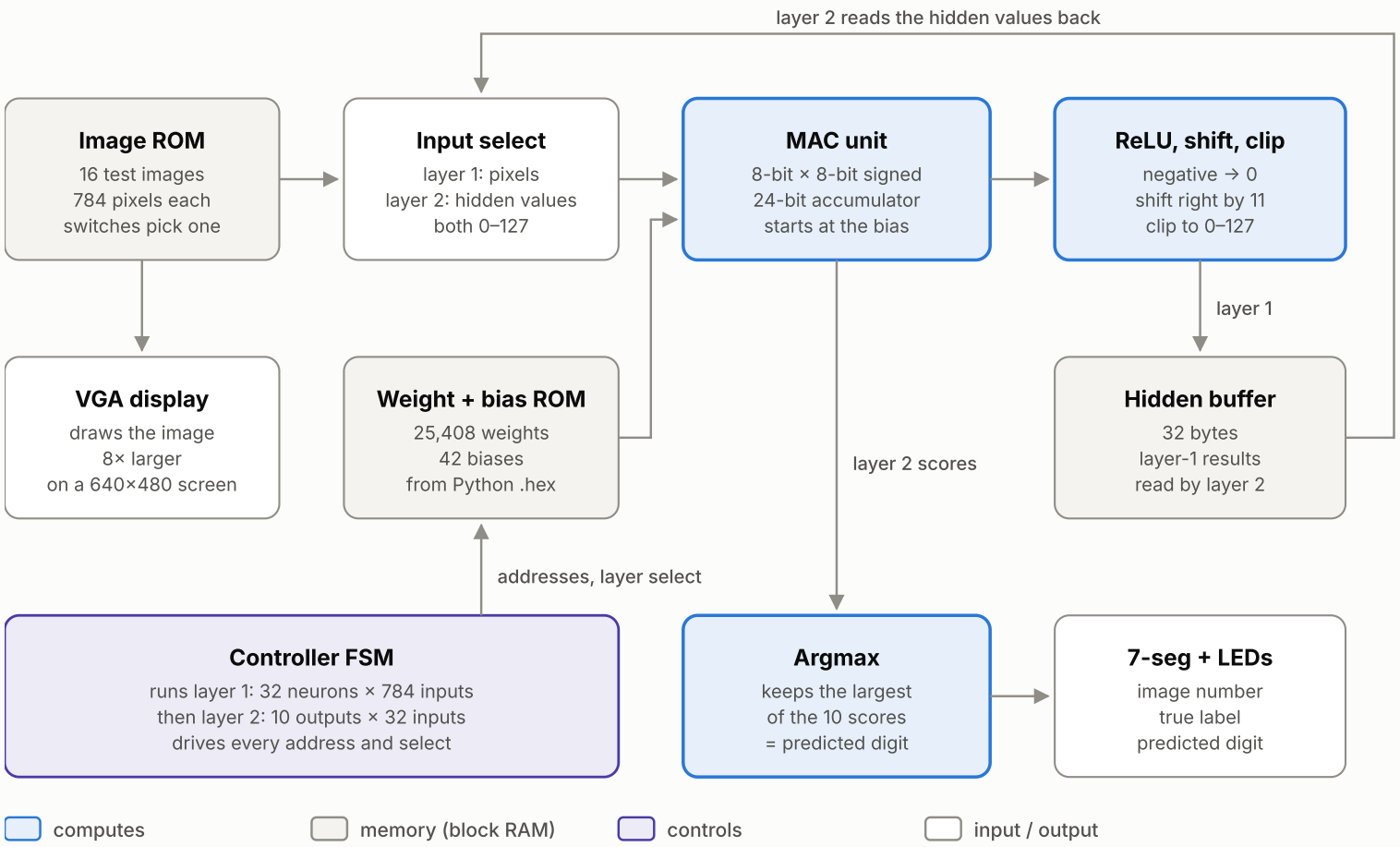




HARDWARE ARCHITECTURE

One MAC unit does all the maths, layer by layer

Every block is a small Verilog module with its own testbench. Blue blocks compute, grey blocks store, and the violet controller tells everything when to run.



Layer 1
32 neurons × 784 inputs
25,088 multiply-adds

Layer 2
10 outputs × 32 inputs
320 multiply-adds

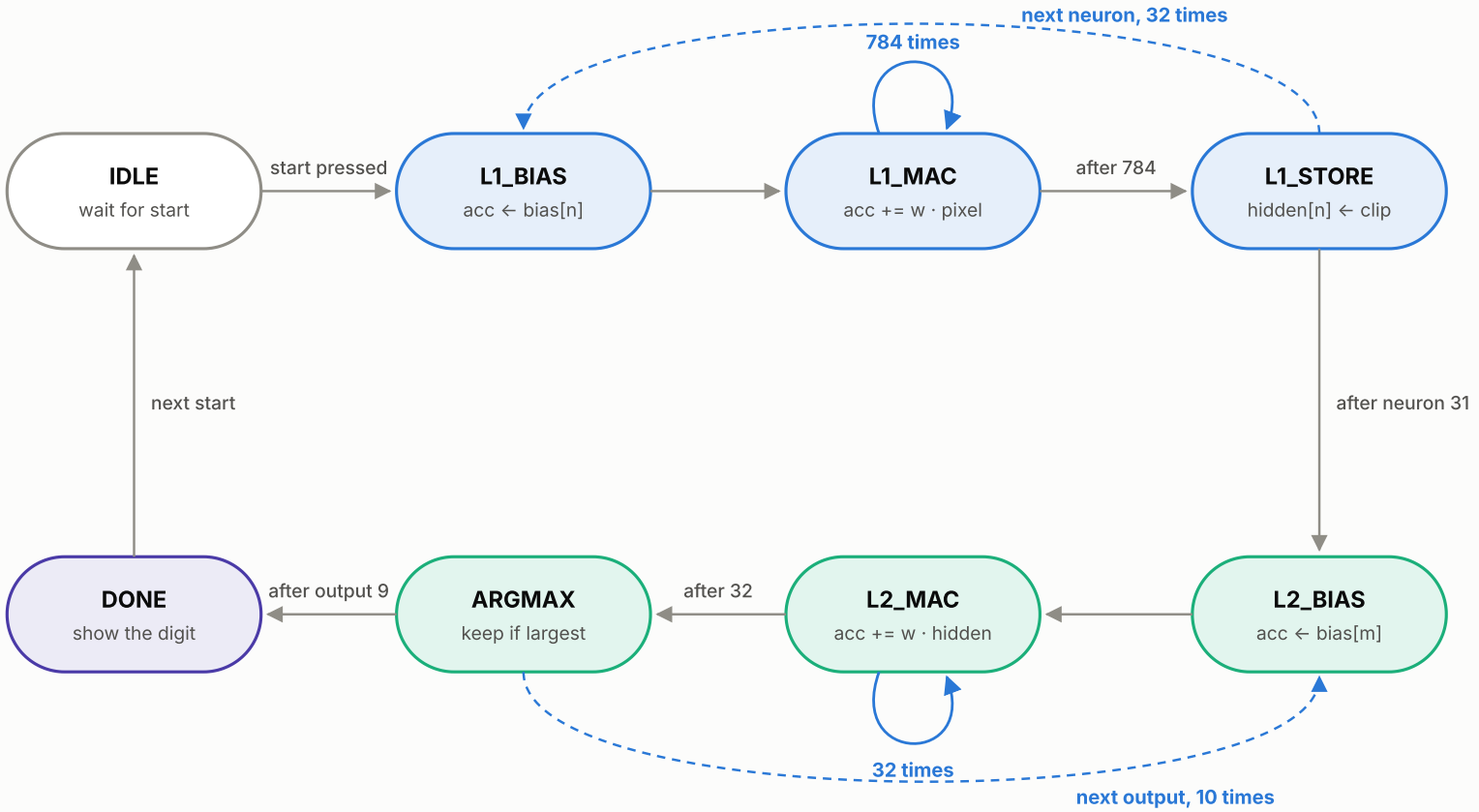
One image
25,408 clock cycles
≈ **254 µs** at 100 MHz

Key idea
The same MAC unit is reused for both layers. Only the inputs and weights change.

CONTROLLER

The controller is one state machine with two loops

Eight states run the whole network. Counters inside the states count inputs (784 or 32) and neurons (32 or 10).



Layer 1 states

Bias, 784 multiply-adds, then store one hidden value. Repeat for all 32 neurons.

Layer 2 states

Bias, 32 multiply-adds, then compare with the best score so far. Repeat for all 10 outputs.

Why it matters

This is a Moore FSM with counters, a core topic of this module. Practise drawing it from memory for the viva.

Watch out

Block RAM data arrives one clock after the address. Delay the accumulate signal by one clock.

MEMORY AND SPEED

Everything fits easily on the board

All weights, biases and test images sit in block RAM inside the FPGA. No external memory is needed.

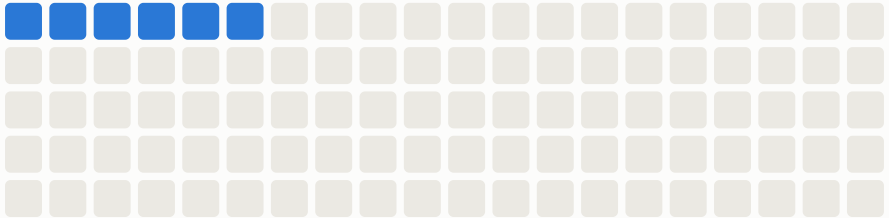
What is stored: 38,068 bytes in total



Weight ROM	25,408 bytes	layer 1: 32 × 784, then layer 2: 10 × 32
Image ROM	12,544 bytes	16 images × 784 pixels
Bias ROM	84 bytes	42 × 16-bit
Hidden buffer	32 bytes	32 bytes

Share of the board's block RAM

about 607 KB on the Artix-7 100T; each square is 1%



≈6%

used

Time to recognise one image at 100 MHz

1 MAC unit (core)



254 µs
25,408 cycles

8 MAC units (stretch)



32 µs
3,177 cycles

Both are instant to a person pressing a button.
The 8× speed-up is a strong number for your resume.

SCOPE

Finish a simple core first, then grow it

The core is what the module marks. Each later step builds on it without changing what already works.

Core

must finish for the module

- 784 → 32 → 10 network in 8-bit
- Python golden model, exact match
- 1 MAC unit, ReLU, argmax, FSM
- 16 images picked by switches
- VGA image + seven-segment answer
- Timing, size and cycle numbers
- Report and viva

Stretch

spare week, only after the core is tagged

- 8 MAC units in parallel, ≈8× faster
- Measured speed-up
- Output scores as a bar chart on the monitor

Later

after the module, for your resume

- Send images from a PC over UART
- Draw digits with a mouse
- Systolic array, then a small CNN
- OpenLane chip layout of the MAC array

Out of scope

not part of this project

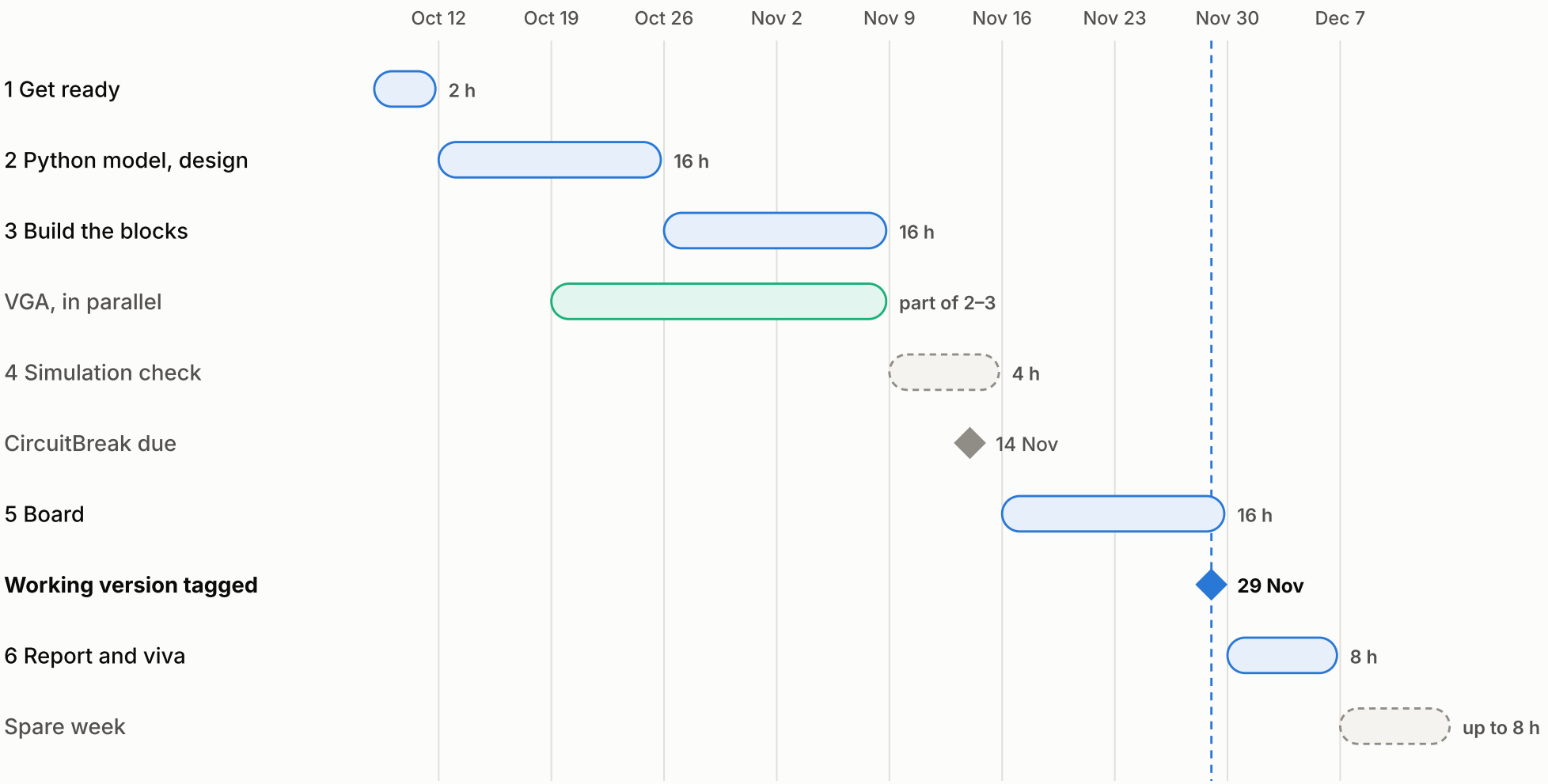
- ~~Training on the FPGA~~
- ~~Camera input~~
- ~~Floating point~~
- ~~A CPU~~

grows step by step after the semester

TIMELINE

Working accelerator tagged on 29 November, with a spare week after

About 8 team hours a week (4 people × 2 hours). The CircuitBreak week is kept light. VGA work runs alongside phases 2 and 3.



PHASES

Six phases, each ending with a gate

A phase starts only when the one before it passes its gate. The spare week (7–13 Dec) catches up anything late, then starts the 8-MAC speed-up.

1 Get ready

before 12 Oct · 2 h

- Lecturer approves the topic
- Agree roles, create the Git repo
- Check Vivado, install Python + NumPy

Gate: topic approved, roles agreed

2 Python model and design

12–25 Oct · 16 h

- Train, convert to 8-bit, golden model
- Export .hex files for weights, biases, images
- Freeze diagram, formats and ports; blink an LED

Gate: model above 95%, .hex files in Git, LED blinks

3 Build the blocks

26 Oct – 8 Nov · 16 h

- MAC, ReLU + shift + clip, argmax
- Memories, counters, controller FSM
- VGA shows a fixed image

Gate: all testbenches PASS; image 0 matches Python

4 Simulation check

9–15 Nov (light week) · 4 h

- Run all 16 images in simulation
- Compare every hidden value and score
- Fix any mismatch

Gate: all 16 images match Python exactly

5 Board

16–29 Nov · 16 h

- Constraints, start button, switches
- Seven-segment, LEDs, VGA image
- Record LUTs, DSP, BRAM, slack, cycles

Gate: board equals simulation; tagged 29 Nov

6 Report and viva

30 Nov – 6 Dec · 8 h

- Each person writes up their part
- Exam-practice tasks
- Explain the whole design to each other

Gate: report done; everyone can explain every block

TEAM

Four roles, one exam-practice task each

The viva is individual, so everyone must be able to explain the whole accelerator. Spend 15 minutes every Friday walking each other through the week's work.

Python and testing

- Training and 8-bit conversion
- Python golden model, .hex export
- Testbenches that compare with Python

Exam practice: Read Vivado's resource report

Compute

- MAC unit
- ReLU + shift + clip
- Argmax; later the 8-MAC version

Exam practice: Write the MAC unit in VHDL

Control and memories

- Controller FSM, address counters
- All memories, top-level module
- Git repo

Exam practice: Moore FSM; setup slack by hand

Board and display

- Constraints, debounce
- Seven-segment driver, LEDs
- VGA controller

Exam practice: VGA timing as counters + FSM

Who leads in each phase

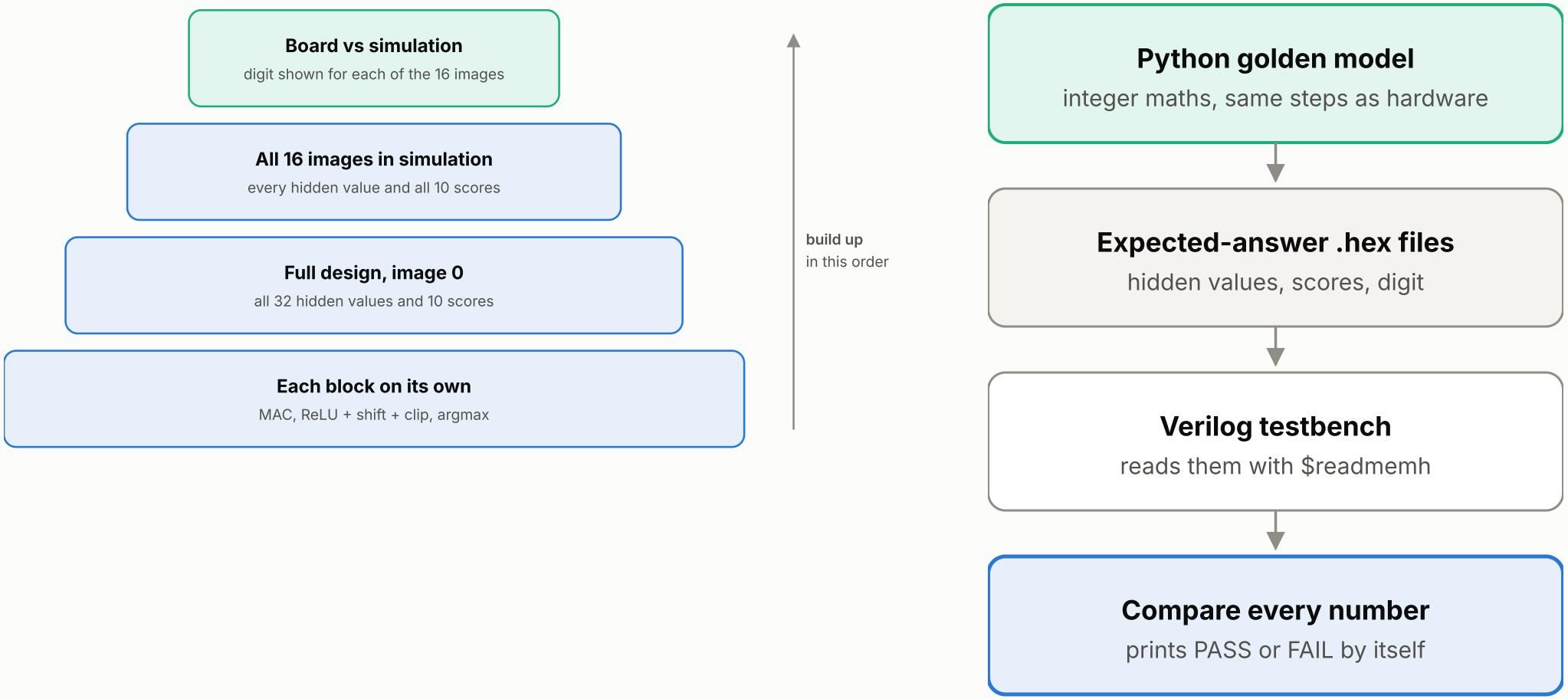
	2 Python, design	3 Build	4 Sim check	5 Board	6 Report
Python and testing	●	●	●	○	●
Compute	○	●	○	○	●
Control and memories	○	●	○	●	●
Board and display	●	●		●	●

● leads the work ○ helps

TESTING

Python is the referee: the hardware must match it exactly

Matching the final digit is not enough. Every intermediate number must be identical, so the first wrong step is easy to find.

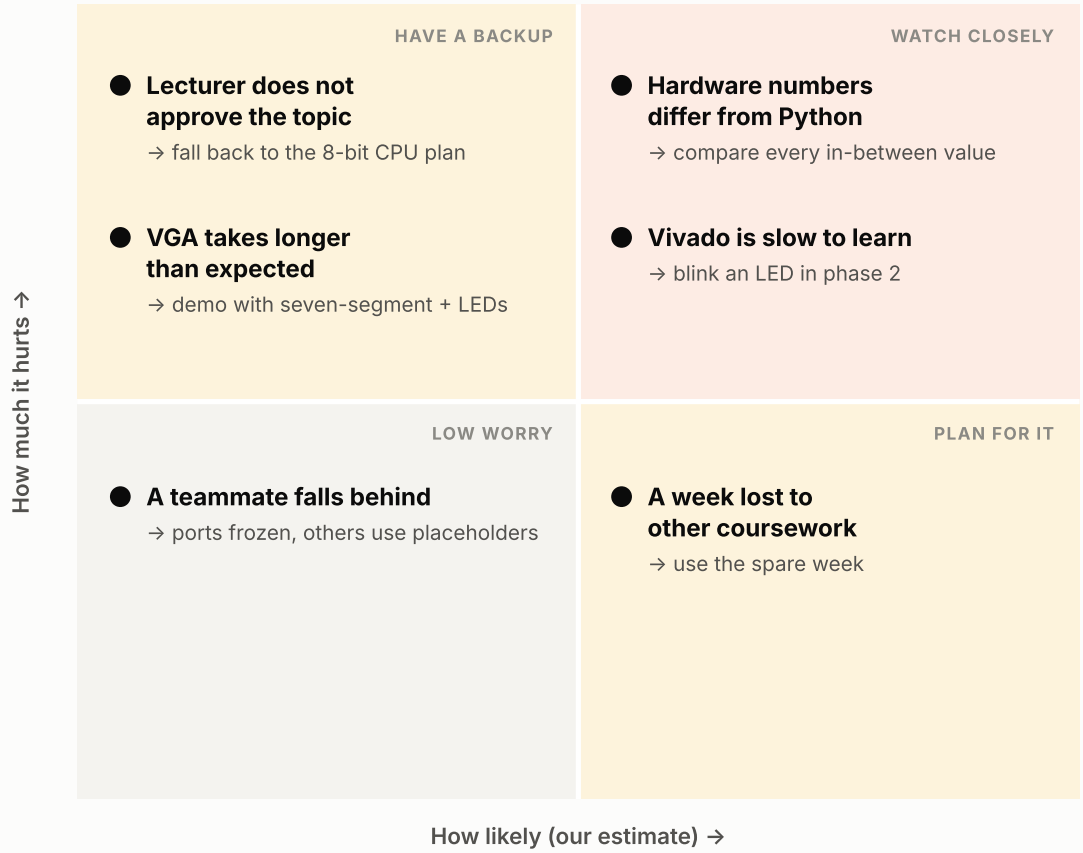


RISKS AND BUGS

The biggest risk is small number mismatches, so test every step

Risk positions are our estimate. Each risk has a ready answer.

Six risks and the answer to each



Seven bugs to avoid

1. Block RAM data comes one clock after the address: delay the accumulate signal
2. Mark MAC inputs and accumulator signed; mixing signed and unsigned makes it all unsigned
3. Sign-extend the 16-bit bias to 24 bits before loading the accumulator
4. Weight for neuron n, input i is at address n × 784 + i, in Python and Verilog alike
5. Python must shift right by 11 and clip to 127, exactly like the hardware
6. Add every .hex file to the Vivado project, or the memory is silently empty
7. Invert CPU_RESETN: the reset button is active-low

CHECKLISTS

Print this page and tick it off

The left list comes before 12 October. The right list is how you know the project is finished.

Before you start

- [ ] Lecturer approves the topic
- [ ] Ask for the demo date and marks split
- [ ] Agree the 4 roles; create the Git repo
- [ ] Check lab PCs for Vivado and Icarus Verilog
- [ ] Install Python 3 with NumPy
- [ ] Check a VGA monitor and cable are available
- [ ] Download Digilent's Nexys-A7-100T-Master.xdc

Done when

- [ ] Python 8-bit model above 95% on 10,000 test images
- [ ] MAC, ReLU + shift + clip, argmax testbenches PASS
- [ ] Simulation matches Python for all 16 images
- [ ] Board matches simulation for all 16 images
- [ ] VGA image + seven-segment answer working
- [ ] No latches; timing met at 100 MHz
- [ ] Recorded LUTs, flip-flops, DSP, BRAM, slack, cycles
- [ ] Working version tagged in Git
- [ ] Report: diagram, formats, ports, tests, README
- [ ] Everyone can explain the whole accelerator